Elena Song
BIOS 702
Exercise 4.1
2026-10-05

```
library(tidyverse)
library(ggplot2)

ultra <- read.csv("ultrarunning.csv")
```

Question 1

Remember from Exercise 1.1 that we are interested in quantifying the effect of emotional intelligence (teique_sf) on ultramarathon times (pb100k_dec). To do so, we will first apply the matrix version of the SLR model.

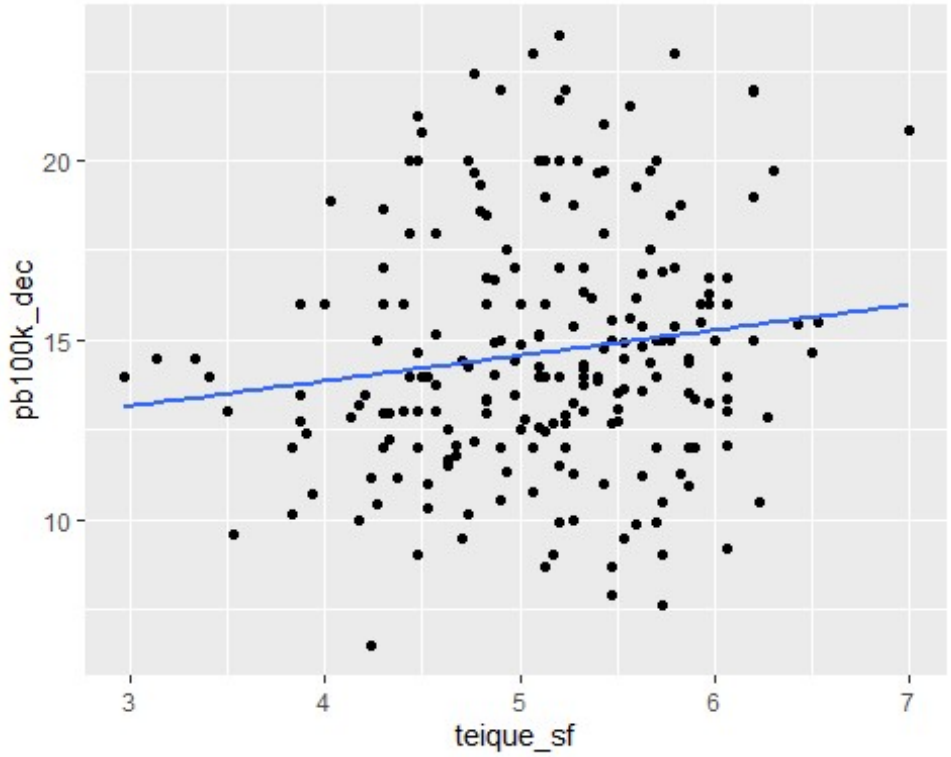
1. Prepare your data. Select only the variables you need and remove any observations with missing data for the purposes of this analysis. We typically add a column of 1's (when doing the calculations "by hand" like this) to represent the intercept. Append a column of 1's to your data.

```
#prepare data
ultra_slr = ultra %>%
  mutate(intercept = 1) %>%
  select(intercept, teique_sf, pb100k_dec) %>%
  na.omit()
```

2. We already looked at univariate distributions of these variables in previous exercises. Now, make a scatterplot of personal best 100k times in hours vs emotional intelligence score. Append a best fit line.

```
#scatter plot
ggplot(data = ultra_slr,
       aes(x = teique_sf, y = pb100k_dec)) +
  geom_point() +
  geom_smooth(method = lm, se = FALSE)

## `geom_smooth()` using formula = 'y ~ x'
```



3. Transform your data into a matrix X (which should be of dimension 2 × n) and a vector Y. Thenmultiply (X'X)−1 by X'Y to calculate the parameter estimates from the SLR model. What are they?

```
#matrix X
X = as.matrix(ultra_slr[,1:2])

#matrix/vector Y
Y = as.matrix(ultra_slr[,3])

#beta hat = (X'X)^-1X'Y
beta_hat = solve(t(X) %*% X) %*% t(X) %*% Y
beta_hat
```

```
##                [,1]
## intercept 11.033815
## teique_sf  0.706835
```

The estimated intercept is 11.034, and the estimated slope is 0.707.

Question 2

1. Now run a SLR on your data using the lm() function (note you no longer need the intercept column).Then run the summary() function. Among others, a table of parameter estimates is printed. Do these parameter estimates match what you obtained using matrix operations?

```
ultra_lm = lm(pb100k_dec ~ teique_sf,
              data = ultra_slr)

summary(ultra_lm)
```

```
##
## Call:
## lm(formula = pb100k_dec ~ teique_sf, data = ultra_slr)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -7.5237 -2.1808 -0.4426  1.8613  8.7906
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  11.0338     1.7318   6.371 1.14e-09 ***
## teique_sf     0.7068     0.3348   2.111   0.0359 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.403 on 213 degrees of freedom
## Multiple R-squared:  0.02049,    Adjusted R-squared:  0.01589
## F-statistic: 4.456 on 1 and 213 DF,  p-value: 0.03594
```

Yes. The parameter estimates from lm() match the estimates obtained using the matrix calculation.

2. Run the names() function on your lm object to find the names of its components. How many components do you see?

```
names(ultra_lm)
```

```
##  [1] "coefficients"  "residuals"     "effects"       "rank"
##  [5] "fitted.values" "assign"        "qr"            "df.residual"
##  [9] "xlevels"       "call"          "terms"         "model"
```

There are 12 components in the lm object.

3. Print the 'coefficients' component. What does the output represent?

```
ultra_lm$coefficients
```

```
## (Intercept)   teique_sf
##   11.033815    0.706835
```

The output represents the estimated regression coefficients. The estimated intercept is 11.034 and the estimated slope for teique_sf is 0.707.

4. In R code, how would you refer to the element of the 'coefficients' component that contains β1

```
ultra_lm$coefficients[2]
```

```
## teique_sf
##  0.706835
```

ultra_lm$coefficients[2] refers to the estimated slope, beta1.

5. Using the information from names(), assign the fitted values to a new object and print the first 5 fitted values.

```
Fitted = ultra_lm$fitted.values
head(Fitted, 5)
```

```
##        2        3        4        5        6
## 15.08398 14.80125 14.80125 14.80125 14.73056
```

Fitted values are the predicted values of the outcome from the regression model.

6. Verify that the components of Fitted are identical to the result of applying the predict() function to your lm object.

```
head(Fitted)
```

```
##        2        3        4        5        6        7
## 15.08398 14.80125 14.80125 14.80125 14.73056 15.25362
```

```
head(predict(ultra_lm))
```

```
##        2        3        4        5        6        7
## 15.08398 14.80125 14.80125 14.80125 14.73056 15.25362
```

```
all.equal(Fitted, predict(ultra_lm))
```

```
## [1] TRUE
```

Yes. The fitted values are equal to the values returned by predict().

7. Check the calculation of the first fitted value, using the first participant's emotional intelligence and the estimated coefficients. Does the calculation match the first fitted value?

```
ultra_lm$coefficients[1] +
  ultra_lm$coefficients[2] * ultra_slr$teique_sf[1]
```

```
## (Intercept)
##    15.08398
```

```
Fitted[1]
```

```
##        2
## 15.08398
```

Yes. The manually calculated fitted value matches the first fitted value from the model.

Question 3

Now we will generate the inputs to the ANOVA table.

1. The first task is to collect Y,Ym, and Yp, all as vectors. You already have Y and Yp. Calculate the mean of Y and copy the results into a vector using the rep() function.

```
Ym = rep(mean(Y), length(Y))
Yp = predict(ultra_lm)
```

2. Create a vector that (1) calculates Y −Ym; and then (2) squares the results. Sum the elements of this vector, and name the result SST.

```
#SST - deviation of obs from mean
SST = sum((Y - Ym)^2)
SST
```

```
## [1] 2518.397
```

3. Create a vector that (1) calculates Y −Yp; and then (2) squares the results. Sum the elements of this vector, and name the result SSE.

```
#SSE - deviation of obs from predictied
SSE = sum((Y - Yp)^2)
SSE
```

```
## [1] 2466.788
```

4. Create a vector that (1) calculates Yp −Ym; and then (2) squares the results. Sum the elements of this vector, and name the result SSR.

```
#SSR -- deviation of predicted from mean
SSR = sum((Yp - Ym)^2)
SSR
```

```
## [1] 51.60908
```

5. You have now calculated the 3 sums of squares in the ANOVA table. What are their values?

```
SST
```

```
## [1] 2518.397
```

```
SSE
```

```
## [1] 2466.788
```

```
SSR
```

```
## [1] 51.60908
```

SST = 2518.397, SSE = 2466.788, and SSR = 51.609.

6. Generate an ANOVA table directly from your lm object by applying the anova() function. Do you obtain the same sums of squares? As a default, R doesn't print SST. From a statistical perspective, why not?

```
#anova table
anova(ultra_lm)
```

```
## Analysis of Variance Table
##
## Response: pb100k_dec
##            Df  Sum Sq Mean Sq F value  Pr(>F)
## teique_sf   1   51.61  51.609  4.4563 0.03594 *
## Residuals 213 2466.79  11.581
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Yes. The ANOVA table gives the same SSR and SSE values as the manual calculations. SST is not printed because it is redundant; it can be obtained by adding SSR and SSE.

7. How could you use the output object from the anova() function to calculate SST? Verify that your code works.

```
anova_table = anova(ultra_lm)

sum(anova_table$`Sum Sq`)
```

```
## [1] 2518.397
```

```
all.equal(SST, sum(anova_table$`Sum Sq`))
```

```
## [1] TRUE
```

SST can be calculated by summing the Sum Sq values from the ANOVA table. The result matches the manually calculated SST.

Question 4

The variance-covariance matrix contains the variance of β0, the variance of β1, and the covariance between β0 and β1. We'll focus on the variance of β1.

1. The vcov() function calculates the variance-covariance matrix directly. Apply this function to your lm object. What is the value of the variance of β1? Take the square root of the variance of β1. Does it equal the standard error of the slope coefficient from the output summary in Question 2.1?

```r
# variance-covariance matrix
ultra_vcov = vcov(ultra_lm)
ultra_vcov
```

```
##             (Intercept)  teique_sf
## (Intercept)   2.9989739 -0.5746213
## teique_sf    -0.5746213  0.1121146
```

```r
# variance of beta1
var_b1 = ultra_vcov[2,2]
var_b1
```

```
## [1] 0.1121146
```

```r
# standard error of beta1
se_b1 = sqrt(var_b1)
se_b1
```

```
## [1] 0.3348352
```

The variance of beta1 is 0.1121, and its standard error is 0.3348.

2. Nowconsider the algebraic formula for V ar(β1) : {∑i(Yi−Yp)2/(n−2)}/∑ i(Xi−Xm)2. The standard error is the square root of this. You have already created most of the elements of this formula. Verify that this formula yields an identical estimate of se(β1).

```r
# mean of X
Xm = mean(ultra_slr$teique_sf)

# calculate SE of beta1 using the algebraic formula
se_b1_formula = sqrt(
  (sum((Y - Yp)^2) / (length(Y) - 2)) /
  sum((ultra_slr$teique_sf - Xm)^2)
)

se_b1_formula
```

```
## [1] 0.3348352
```

```r
all.equal(se_b1, se_b1_formula)
```

```
## [1] TRUE
```

The standard error calculated using the algebraic formula is 0.3348, which is identical to the standard error obtained from vcov().

3. The formula contains 3 terms. Under what circumstances will the ∑i(Yi − Yp)2 term be small? Consider, in particular, what happens to this term when the model fits well vs poorly?
   The sum of squared differences between the observed and predicted values will be small when the model fits the data well, because the observed values are close to the predicted values.

4. Underwhatcircumstances will the ∑i(Xi−Xm)2 term be large? When you are designing an experiment and have the ability to assign the values of X, should they have a little variation or a lot of variation?
   The sum of squared differences between X and its mean will be large when X has a lot of variation. When designing an experiment, more variation in X helps estimate the slope more precisely.

Question 5

Now, consider the null hypothesis β1 = 0. We know that we can test this hypothesis using a t-test or an F-test.

1. Run the t-test "by hand", using the formula. Do the t-statistic and p-value match what you found from lm summary in Question 2.1?

```
# t-test
t = beta_hat[2] / se_b1
t

## [1] 2.110994

# two-sided p-value
p_t = 2 * pt(abs(t), df = length(Y) - 2, lower.tail = FALSE)
p_t

## [1] 0.03593904
```

The t-statistic is 2.111 and the two-sided p-value is 0.0359. These values match the results from the lm() summary.

2. Run the F-Test "by hand", using the formula. Do the F-statistic and p-value match what you found from the anova() function in Question 3?

```
# F-test
f = (SSR / 1) / (SSE / (length(Y) - 2))
f

## [1] 4.456294

p_f = pf(f, 1, length(Y) - 2, lower.tail = FALSE)
p_f

## [1] 0.03593904
```

The F-statistic is 4.456 and the p-value is 0.0359. These values match the results from the ANOVA table.

3. Verify that the F-statistic is the square of the t-statistic

```
t^2
```

```
## [1] 4.456294
```

```
f
```

```
## [1] 4.456294
```

```
all.equal(as.numeric(t^2), as.numeric(f))
```

```
## [1] TRUE
```

The F-statistic is equal to the square of the t-statistic: F = t^2 = 4.456.

4. Is there statistical evidence to conclude that emotional intelligence has an effect on ultramarathon times? (Assume a critical value $\alpha = 0.05$). Interpret the slope estimate in the context of the problemas well.
   There is statistically significant evidence of an association between emotional intelligence and ultramarathon time at the 0.05 significance level. For each 1-point increase in emotional intelligence score, the expected ultramarathon time increases by approximately 0.707 hours.

5. Do you think the effect is clinically significant?
   Although the association is statistically significant, the clinical significance cannot be determined from this analysis alone because no threshold for a clinically meaningful difference is provided.